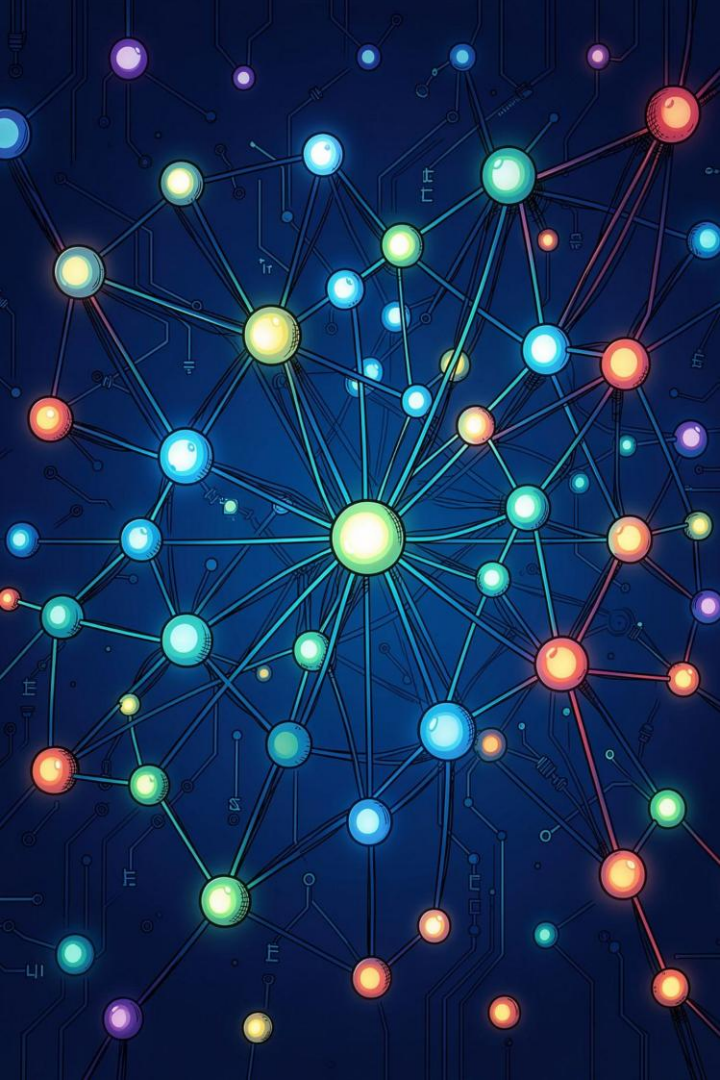




Unification & Inference

Forward Chaining, Backward Chaining & Resolution —
the core reasoning engines of first-order logic.

Presented by:

T.S.Vidhushini [192511061]

K.S.Dharshana[192412015]

G.Kaviya[192524281]

Guided by:

Dr.P.Rajaram

UNIFICATION EXAMPLE

What is Unification?

- Unification is the process of making two logical expressions identical.
- It finds suitable values for variables.
- These values are called **substitutions**.
- After applying the substitution, both expressions become exactly the same.



Real-Time Example

In a social media application:

User searches: Likes(Ravi, IceCream)

System has data: Likes(x, IceCream)

Here, x represents any user.

To match the search, system finds
 $x = \text{Ravi}$.

So, the system understands that
Ravi likes IceCream.



Example:

Expression 1 (General)	Expression 2 (Specific)
Likes(x , IceCream)	Likes(Ravi , IceCream)



Step-by-Step Unification

1. Compare both expressions.
2. Both have same predicate: Likes.
3. First argument is variable 'x' in Expression 1 and constant 'Ravi' in Expression 2.
4. Substitute $x = \text{Ravi}$.
5. After substitution:

Likes(**x**, IceCream)
(After $x = \text{Ravi}$)



Likes(Ravi, IceCream)



Result: Both expressions are now identical: Likes(Ravi, IceCream)
Therefore, the two expressions are successfully unified.

Unification: A Worked Example

Unification finds substitutions that make two first-order logic terms **identical**. It is the matching step that powers inference.

Successful unification

Likes(john, X) and Likes(john, pizza)

Unification failure

Parent(X, Y) and Parent(john, john, mary)

1) Successful example

Likes(john, X) and Likes(john, pizza)

Terms

Likes(john, X)
Likes(john, pizza)

Step-by-step

1. Compare the predicate names: Likes vs Likes → match.
2. Compare first arguments: john vs john → match.
3. Compare second arguments: X vs pizza → bind X to pizza.
4. Apply the substitution everywhere: the terms now become identical.

Substitution: $\theta = \{X/pizza\}$

After applying θ , both expressions evaluate to Likes(john, pizza).

2) Why the algorithm succeeds

01

Decompose

Break each expression into predicate and arguments.

03

Bind

When a variable meets a constant, create a substitution.

02

Compare

Check symbols position by position.

04

Verify

Confirm the substitution makes both terms exactly the same.

3) Failure example

Parent(X, Y) and Parent(john, john, mary)

Terms

Parent(X, Y)
Parent(john, john, mary)

Why it fails

- Parent(X, Y) has **2 arguments**.
- Parent(john, john, mary) has **3 arguments**.
- Unification requires the same predicate symbol *and* the same arity.
- Because the arity differs, no substitution can make them identical.

⚠ **No unifier exists.** This failure happens before any variable binding can help.

LIFTED FORWARD CHAINING

? What is Lifted Forward Chaining?

- Forward chaining starts with **known facts**.
- It applies rules to generate **new facts**.
- It continues until the required **goal** is found.
- Lifted means it works with **variables** instead of checking every individual case.
- It is also called **data-driven reasoning**.

★ Real-Life Example

Healthcare Diagnosis System

A doctor uses general medical rules and patient information to infer new conclusions.

-  **Fact:** Patient: Ravi
Fact: Fever(Ravi)
-  **Rule:** Fever(x) → Flu(x)
Flu(x) → NeedRest(x)
-  **New Fact:** Flu(Ravi)
NeedRest(Ravi)

 **Goal:** NeedRest(Ravi) (Achieved)



Simple Example

	Fact (Known):	Human(Ravi)
	Rule:	Human(x) → Mortal(x)
	New Fact (Derived):	Mortal(Ravi)

How it Works (Step-by-Step)



Conclusion:

Lifted forward chaining uses general rules with variables to infer new facts efficiently, without checking every single individual case.

Forward Chaining: Worked Example

We start with two facts and one rule, then repeatedly match, fire, and add new facts until no more conclusions can be derived.

Initial KB

Facts

```
Likes(john, pizza)
Likes(mary, wine)
```

Rule

```
If Likes(X, Y) then Enjoys(X, Y)
```

1

2

Match

Unify the rule premise with `Likes(john, pizza)`.

Fire

Use $\theta = \{X/\text{john}, Y/\text{pizza}\}$ to instantiate the conclusion.

3

4

Add

Add `Enjoys(john, pizza)` to the knowledge base.

Repeat

Match the same rule with `Likes(mary, wine)`, then fire again.

Final KB

```
Likes(john, pizza)
Likes(mary, wine)
Enjoys(john, pizza)
Enjoys(mary, wine)
```

Forward Chaining Efficiency: Naive vs Incremental

Both methods apply the same rules — the difference is how much information they re-check.

● Naive Forward Chaining

“Check Everything Again”

Knowledge Base: 100 Parent facts

Round 1 → 100 checks

Round 2 → 100 checks again

Round 3 → 100 checks again

Total = $100 \times 3 = 300$ checks

● Incremental Forward Chaining

“Check Only What Is New”

Suppose only 5 new facts are added

Keep previously processed information

Focus on the 5 new/relevant facts

Example: 100 checks → 5 relevant checks

Less repeated work

KEY IDEA: Naive = check everything again | Incremental = continue from what is new

Real-Time Example: College Attendance

A simple analogy makes the efficiency difference easy to understand.

Naïve Approach

A new student enters the classroom.

The teacher checks all 100 students again.

- ✗ Repeats work
- ✗ Takes more time

Incremental Approach

A new student enters the classroom.

The teacher records only that student's attendance.

- ✓ Avoids repeated work
- ✓ Saves time

Forward chaining works the same way: process new information instead of repeatedly scanning everything.

Backward Chaining: Start at the Goal

Facts: Likes(john, pizza)
Rules:
1. If Likes(X, Y) then Enjoys(X, Y)
2. If Enjoys(X, Y) then Happy(X)
Query: Happy(john)?

01

Goal: Happy(john)

Start with the query and search for a rule whose conclusion matches the goal.

03

Unify the variables

 $\theta = \{X/\text{john}\}$

The new subgoal becomes Enjoys(john, Y).

05

Reach the base fact

Substitute again to get Likes(john, pizza), which matches the known fact.

Proof tree

Happy(john)
└ Enjoys(john, pizza)
 └ Likes(john, pizza)

02

Match the conclusion

Find Happy(X) $\leftarrow$ Enjoys(X, Y) and unify Happy(john) with Happy(X).

04

Find the next rule

Match Enjoys(X, Y) $\leftarrow$ Likes(X, Y) and unify it with Enjoys(john, Y).

06

Backtrack and confirm

The proof succeeds, so Happy(john) is true.

Resolution: Worked Example

KB: Likes(john, pizza), If Likes(X, Y) then Enjoys(X, Y)

Query: Enjoys(john, pizza)?

01

1. Convert to CNF

Transform $\text{KB} \wedge \neg \text{Enjoys}(\text{john}, \text{pizza})$ into conjunctive normal form.

03

3. Resolve the rule

Unify $\text{Likes}(\text{john}, \text{pizza})$ with $\text{Likes}(X, Y)$ using $\theta = \{X/\text{john}, Y/\text{pizza}\}$.

05

5. Resolve with negation

Resolve $\text{Enjoys}(\text{john}, \text{pizza})$ with $\neg \text{Enjoys}(\text{john}, \text{pizza})$ using $\theta = \{\}$.

07

7. Conclusion

$\text{Enjoys}(\text{john}, \text{pizza})$ is proven.

Key insight: Each resolution step uses the most general unifier to connect matching literals and move the proof forward.

02

2. Show CNF clauses

$\text{Likes}(\text{john}, \text{pizza})$
 $\neg \text{Likes}(X, Y) \vee \text{Enjoys}(X, Y)$
 $\neg \text{Enjoys}(\text{john}, \text{pizza})$

04

4. Derive the goal

Apply θ to get $\text{Enjoys}(\text{john}, \text{pizza})$.

06

6. Empty clause

Derive $\square$.

A large, irregular splash of teal and blue watercolor paint serves as the background for the text. The colors vary in intensity, with darker blues in the center and lighter teals towards the edges, creating a soft, painterly effect.

Thank You